PAD: Patch-Agnostic Defense against Adversarial Patch Attacks

Maya Geva

Adversarial Patch Attacks

- Attackers place an adversarial patch (localized, visible perturbations) onto a physical object or digital image to cause a model to misclassify it
- Can lead to consequences such as a self driving car not detecting a stop sign
- Unrealistic to change every pixel of an image or cover all of an object, attackers change a limited area
- Patches such as localized noise, printable patch, or a natural looking patch

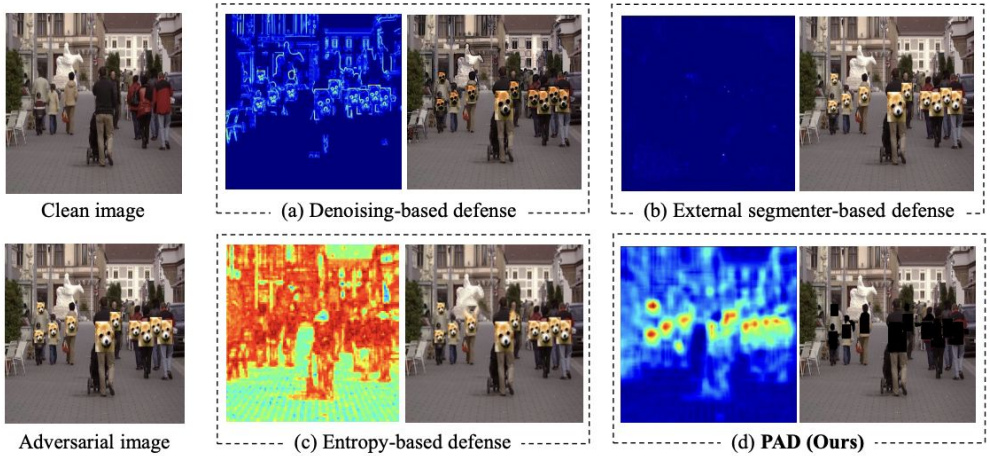
Novelty

Previous Defenses:

- **Denoising-based**: Removes all noise or misses natural looking patches
- **External segmentor-based defenses**: relies on training data and existing attacks
- **Entropy-based defenses**: relies on training data and existing attacks

PAD:

- Works on any patch type, color, location, amount (not image-wide noise, semi-transparent patches, or adaptive attacks)
- Doesn't rely on prior attack knowledge: car idenitfy zero-day attacks
- Doesnt rely on training data, no retraining needed
- Agnostic to patch appearance, shape, size location, quantity
- Preforms better than existing defenses



PAD Method

Semantic Independence Localization: patches are semantically independent from surrounding area (measured with MI heatmap)

Spacial Heterogeneity Localization: the image quality of patch differs from surrounding area. Compressing will affect each quality image differently, showing where patches may be (measured with CD heatmap)

1. INPUT IMAGE (5000×5000) - Resize to 2048×2048
2. Generate MI + CD heatmaps and Fuse Heatmaps: creates patch localization map
3. SAM Segmentation: generate object masks
4. IOU Matching: consider all masks with an IoA > threshold a patch attack
5. Remove Patches: black out patch pixels

Step 1 - Image Resizing

I got an error with trying to process the 5000x5000 images

GPU memory needed: ~17.9 GB

GPU available: 11.9 GB

Solution:

- Resized to 2048×2048
- GPU memory needed: 3-4 GB
- 16x less memory
- Processing speed: ~2-3 min per image

Step 2 - Heatmap Generation

Mutual Information (MI) Heatmap:

- What it does: Detects anomalies using entropy
- How: Measures information change at each pixel
- Result: High values where patches exist

Change Detection (CD) Heatmap:

- What it does: Detects region differences
- How: Compares regions looking for changes
- Result: High values where images differ

Fusion:

- MI + CD ➜ Combined heatmap
- Highlights regions that may contain patches

Step 3: SAM Segmentation

Segment Anything Model (SAM):

- Pre-trained vision transformer, no labels needed
- Generates object masks from image
- No fine-tuning needed

Step 4: Patch Detection

For each SAM mask:

- Calculate IOU with heatmap region
- IOU = Overlap / Total Area

Decision Logic:

$$IoA(mask, H_p) = \frac{area(mask \bigcap H_p)}{area(mask)}.$$

IF IOU > threshold (80%)

- Patch detected, black out every pixel in the mask

Results from original code

- I tested on images from the INRIA Arial Images Database
- Generated 2 patches per image (red squares + noise)
- PAD covered noise patches but not red squares

Attacked Image

Image defended with PAD

My Improvement

The original PAD method uses the heatmap to look for entropy (random changes), but didnt pick up on the solid red patches

Add Color-Based Detection: look for very saturated colors (patch indicator)

1. Convert to hsv: has the hue, saturation, and value of each pixel
2. Apply a threshold of 150 (out of 255) meaning the pixel is more saturated than a natural scene would likely have
3. Combine with the heatmap method. If either method detects a patch, remove it

My improvement results & limitations

Successfully removed solid color patches from the images

Limitation:

- Only works on solid color patches
- Might have false positives on other image types (part of image is a solid color but isn’t a patch)

Attacked image

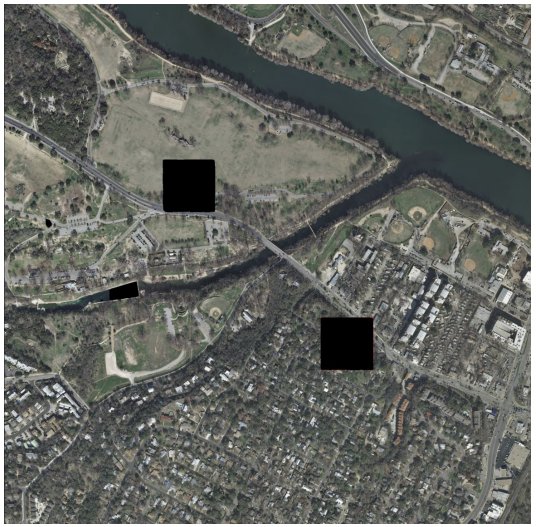


Image defended with PAD + my improvement

Limitations and Future Improvements

Current Limitations:

- Only tested on aerial images
- Requires GPU (not real-time on CPU)
- Slow (2-3 min per image)
- Only tested on specific patch types (red, noise)

Future Improvements:

- Detect patches at different scales
- Evaluate on more diverse patch types
- Evaluate on different datasets